

Research article



Server load and network-aware adaptive deep learning inference offloading for edge platforms

Jungmo Ahn^{a,b}, Youngki Lee^c, Jeongseob Ahn^{a,*}, JeongGil Ko^{d,*}

^a Department of Artificial Intelligence, Ajou University, 206 Worldcup-ro, Yeongtong-gu, Suwon 16499, South Korea

^b Hyundai MOBIS, 203 Teheran-ro, Gangnam-gu, Seoul, 06141, South Korea

^c Department of Computer Science and Engineering, Seoul National University, 1 Gwanak-ro, Gwanak-gu, Seoul, 08826, South Korea

^d School of Integrated Technology, Yonsei University, 50 Yonsei-ro, Seodaemun-Gu, Seoul, 03722, South Korea

ARTICLE INFO

Keywords:

Computation offloading
Deep neural networks
Distributed systems
Mobile/embedded computing

ABSTRACT

This work presents *DIAMOND*, a deep neural network computation offloading scheme consisting of a lightweight client-to-server latency profiling component combined with a server inference time estimation module to accurately assess the expected latency of a deep learning model inference. Latency predictions for both the network and server are comprehensively used to make dynamic (partial) model offloading decisions at the client in run-time. Compared to previous work, *DIAMOND* targets to minimize network latency estimation overhead and considers the concurrent processing nature of state-of-the-art deep learning inference server designs. Our extensive evaluations with an NVIDIA Jetson Nano client connected to an NVIDIA Triton server shows that *DIAMOND* completes inference operations with noticeably reduced computational/energy overhead and latency compared to previously proposed model offloading approaches. Furthermore, our results show that *DIAMOND* well-adapts to practical server load and network dynamics.

1. Introduction

With growing demands to employ large deep neural networks (DNNs) for mobile and IoT applications, techniques focusing on offloading complex computations from resource-limited mobile and embedded devices to high-performance cloud servers has gained noticeable attention [1–4]. As an attractive collaborative system architecture for mobile deep learning inference compared to a fully on-device approach, computation offloading possess several advantages of (1) enabling DNN operations on low-end devices that lack GPUs or dedicated HW accelerators (e.g., NPUs) and (2) supporting the usability of previously computation-intensive deep learning models (due to complexity) on mobile and embedded platforms. Given that state-of-the-art mobile GPUs (e.g., Qualcomm Adreno 650 [5]) are (at least) an order-of-magnitude limited in terms of processing power compared those used on servers (e.g., NVIDIA RTX 2080Ti [6]), collaboratively exploiting such computing power is an appealing design choice. However, despite active research in deep model offloading techniques that exploit model partitioning, adaptation, and early exit schemes [7–17], minimizing end-to-end inference latency under dynamically changing network conditions and complex server workloads still remains a challenge.

In this paper, we propose *DIAMOND*, an end-to-end system for effective collaborative deep learning via computation offloading. *DIAMOND* efficiently estimates the server and network conditions, which are key factors contributing to the end-to-end inference latency in making on-the-fly model partitioning (and offloading) strategy decisions. Specifically, *DIAMOND* imposes three major

* Corresponding authors.

E-mail addresses: jsahn@ajou.ac.kr (J. Ahn), jeonggil.ko@yonsei.ac.kr (J. Ko).

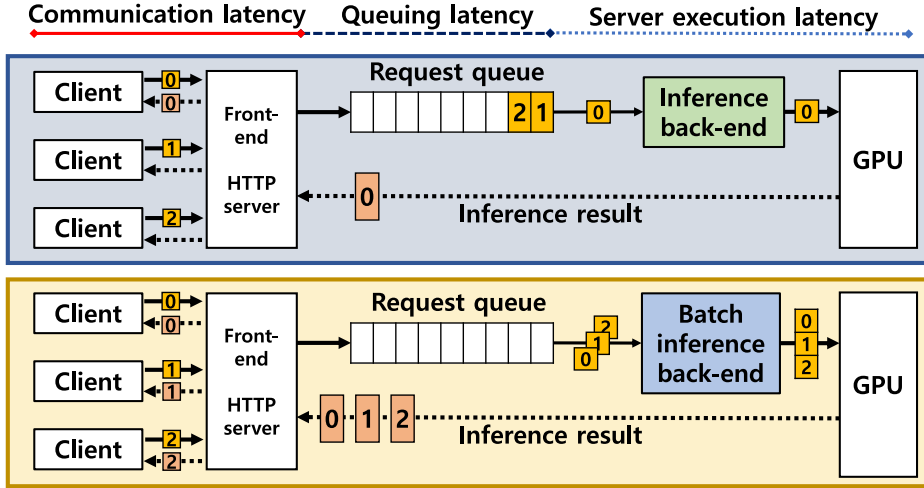


Fig. 1. Operation flow comparison of collaborative deep learning inference for a server configuration with single batch inference (top) and multi-batch inference (bottom).

contributions in comparison with prior deep learning model offloading approaches: (i) providing system-level support for accommodating a commercially available dedicated inference server with multi-batch inference support (e.g., NVIDIA Triton Inference Server [18]) for deep learning model offload operations, (ii) proposing schemes for maximizing server-side parallelism; thus, the inference processing throughput, (iii) designing server and network latency profiling/estimation schemes for precise offloading decision support.

Our extensive evaluations using the NVIDIA Jetson Nano and Triton Inference Server, show that *DIAMOND* can accurately estimate dynamic network and server conditions to make effective model partitioning decisions at the embedded client device. Quantitatively, for sporadic offloading of state-of-the-art DNNs (e.g., EfficientNet) under dynamic network and server conditions, *DIAMOND* shows up to ~30% latency reduction and 38% communication overhead reduction by accurately estimating the network latency and server load compared to previously proposed schemes. Moreover, *DIAMOND*'s server load monitor effectively diagnoses server status with near two magnitudes of lower energy compared to prior work.

[Challenges] Designing *DIAMOND* involves the following technical challenges that were merely considered previously.

- **Operational characteristics of deep learning inference servers.** Recently released dedicated deep learning inference servers such as the NVIDIA Triton are highly optimized to improve the overall inference throughput and latency [18]. They employ techniques such as multi-batch processing to fully exploit the parallel processing, but, in turn, this complicates the server-side inference processing latency estimations [19–21]. Given the complex operational characteristics of these servers, it is important to take these complications in consideration prior to making latency estimations that affect model offloading decisions [22]. Moreover, both queueing and execution latency can dynamically change depending on the server load conditions; thus, it is challenging to predict the inference request processing latency using a pre-defined server load pattern estimation model.

- **Efficiency in resource profiling and offloading decision making.** To support applications that target high user-experience levels (for instance, continuous vision or augmented reality applications), deep learning inference should be made with minimal latency. To maximize the benefits of collaborative inference, offloading decisions need to be made in a highly-efficient manner and the underlying server/network resource profiling schemes should be light-weight. Prior techniques typically exploit network monitoring tools such as iPerf [7,23], which unnecessarily increases the network traffic and thus the end-to-end inference latency. Moreover, to diagnose the server load, prior techniques require the exchange of dummy requests, which adds overhead both in terms of energy and computation cost [13]. Ideally, resource profiling and offloading decision operations should be done with minimal explicit overhead.

[Approach] To tackle these challenges, in *DIAMOND*, we design a suite of novel techniques and highlight them below.

- **Inference request processing latency prediction.**

As Fig. 1 shows, the inference request processing latency at the server consists of the queuing latency and the actual inference (processing) latency. Increased loads at the server (when many clients issue inference requests) lead to higher queuing latency, and with a larger batch size, the inference latency increases. *DIAMOND* considers the individual characteristics of these two types of latencies by exploiting a probabilistic approach for queuing latency estimation and a history-based estimation for the inference latency.

- **TCP-aware network profiler/estimator.** For deep model offloading, it makes sense to select layers that require minimal transmission to suppress network-induced latency. In fact, for many widely-used models, the amount of data exchange at intermediate layers

is <1000 kB, suggesting that network latency estimations should be fine-grain and at a per-TCP congestion window size level, since transmissions will not experience the maximum TCP window size in most cases. *DIAMOND* integrates network profiling operations with previous server–client packet exchanges and breaks down the expected latency to a TCP congestion window size-level to perform TCP simulation-based network latency estimations.

[Contributions] This work contributes the following.

- We present *DIAMOND*¹ a computation offloading approach for resource-limited edge platforms considering characteristics of modern deep learning inference servers. We consider the dynamic inference patterns that these servers can support, which complicates latency prediction compared to simple server designs adopted in previous work.
- *DIAMOND* exploits a simple, yet accurate network condition estimation module by exploiting TCP operation characteristics, which combined with probabilistic server latency predictions, offers accurate overall inference latency estimations under dynamic operational conditions.
- We propose a layer-centric batch processing scheme which enables efficient collaborative inference on multi-batch inference servers to show improved throughput compared to naive multi-batch approaches and single-batch server designs adopted in previous work. Our evaluations show that this approach improves the server throughput, compared to a naive batching approach, by more than two-fold.

2. Background and related work

We start this work by presenting background information on deep learning model computation offloading and discussing about different deep learning inference server designs. In this section, we also present an overview of some of the most relevant state-of-the-art work.

2.1. Deep learning model computation offloading

Given the computational limitations of mobile and embedded computing platforms, offloading the deep learning model inference operations to a more powerful server is an effective design choice. Such computation offloading allows complex models to (potentially) operate under tight latency constraints and reduce the computational energy usage of resource-limited platforms. Nevertheless, the latency and energy benefits of computation offloading can differ with respect to different network conditions as slower (or unstable) communication link conditions can cause increased transmission delay and attempts. Furthermore, as more client devices connect to a server for computation offloading, the server can face congestion in the processing phase, which also can lead to increased latency (c.f., Fig. 1).

Previously proposed model partition schemes are designed to address such dynamic conditions via a number of different approaches. Mostly, these schemes focus on balancing the DNN computational load between the client and server, or reducing the communication data size by transmitting intermediate output values smaller than the original input data. A representative work, Neurosurgeon introduced the concept of adaptive DNN model partitioning, in which model partitioning takes place with respect to the dynamic network and server conditions [7]. However, Neurosurgeon is only applicable for DNNs of chain topology (e.g., AlexNet [24], VGG16 [25]), whereas DADS [11] and QDMP [26] target more sophisticated topology models (e.g., Inception [27], ResNet [28]). Specifically, these work proposed a fast partitioning point pruning method given many available partitioning points within the model topology.

Many works followed these pioneering work to minimize the inference completion latency of collaborative inference systems. Despite the client sending intermediate output data to reduce the data transmission size, when under harsh network conditions, even this can be too much of a burden. For such reason, DeepCOD adopts an autoencoder-based data compression approach capable of minimizing the communication latency even for extremely limited network conditions [12]. In terms of minimizing the DNN inference latency, SPINN exploits an *early-exit* approach, which reduces both the client- and server-side computation latency while sacrificing some level of classification performance. [13].

Note that for accurate offloading decision making, an accurate network and server processing latency prediction is essential. Typically, collecting such information at the client requires an exploration–exploitation approach [12,13], where clients issue inference requests to the server without any prior knowledge (exploration) then uses this experience to determine the link bitrate and server load. In these schemes, the bitrate of the communication link is computed by dividing the data size with the communication latency [7,13,26]. The server load is estimated empirically compared to an idle-state server condition [13]. Finally, clients make offloading decisions based on such predicted information bits (exploitation).

We make an observation that such network prediction methods are not suitable for DNN computation partitioning. Given that a TCP link is typically used, it is important to acknowledge the slow start characteristics of TCP [29,30]. Without such consideration, we point out in Section 6.2.1 that network latency prediction can be erroneous; leading to a non-optimal partitioning decision.

¹ An open source implementation of *DIAMOND* is available through <https://github.com/eis-lab/diamond>.

2.2. Deep learning inference server designs

State-of-the-art servers dedicated for DNN inference operations adopt multi-batch processing and concurrent processing by exploiting CUDA streams and multi-process services (MPS) supported in recently released GPU architectures [19,21,31–34]. Such parallelism improves GPU utilization and improved inference throughput by sharing GPU resources in both spatial and temporal dimensions. TorchServe [35] and Tensorflow Serving [36] are examples of dedicated deep learning inference server framework supporting multi-batch inference, and the NVIDIA Triton [18] and KFServing [37] offers a multi-batch and concurrent inference environment with multiple execution instances.

In addition to simply exploiting the native capabilities of these servers, there have been various efforts to propose a more effective task scheduling strategy. GSLICE improves the overall GPU utilization by executing multiple inference instances with adaptive resource allocation and spatial scheduling using NVIDIA MPS [19]. Nexus suggests a batching-aware temporal scheduling algorithm inspired by the bin-packing problem to achieve high throughput while satisfying service level objectives (SLO) [21]. Clipper also introduces an adaptive batch inference mechanism to increase server throughput to satisfy SLOs [20].

Despite such advanced server-side operations, we noticed that previously proposed model partitioning-based collaborative deep learning schemes exploit only a simple server design with no batch processing and single instance [7,13]. As a result, they cannot make accurate estimations on server conditions when a more advanced server design is used. In this work, we take in consideration the added complexity of server load estimation when exploiting multi-batch inference designs. For this, we (1) design a novel layer-centric batch inference mechanism to fully exploit the benefits of batch parallelism even with requests issued at heterogeneous points in the model, and (2) propose a probability-based inference latency prediction scheme to adopt to the dynamicness of a multi-batch processing server environment.

3. Preliminary study

We start by conducting a preliminary study to understand factors affecting the performance of collaborative deep learning (DL) inference from an end-to-end perspective. In particular, we leverage the NVIDIA Triton [18], a state-of-the-art GPU-enabled DL inference server, to examine the latency when handling inferences under dynamic computing loads. Furthermore, we examine the effect of network conditions on the end-to-end inference latency between the server and the NVIDIA Jetson Nano client. Based on these experiments, we will discuss opportunities and directions for designing an adaptive framework for collaborative DL.

3.1. Experimental environment

To understand the end-to-end latency of collaborative inference requests, we mimic a real-world DL computation offloading scenario of an embedded image classification system assisted by an externally connected server.

Target DNN model and metrics: Here, we exploit the EfficientNet-B5 model, a state-of-the-art image classification model based on a CNN architecture [38]. We measure per-model layer processing latency (at the client and server) to examine the performance at potential offloading points and decompose the latency into client computation, networking, server queuing, and server computation.

Inference server and client platform: We use the NVIDIA Triton Inference Server [18] with the LibTorch C++ backend [39]. The server is hosted on a machine equipped with an NVIDIA RTX 2080Ti GPU [6] and two socket Xeon Gold 5218 CPUs [40]. Each socket consists of 16 physical cores and 32 GB DRAM. This environment enables multi-batch model execution, which leads to high GPU utilization and throughput. For server load generation, we use the `perf_analyzer`, NVIDIA's performance analysis utility [41], running on a separate machine where multiple threads are used to mimic virtual inference request issuing clients. We note that such a configuration is different from assumptions made in most previous work [7,9,11–13,26]. Specifically, previous work assume a simple server operating a single application with no batch processing. However, we argue that our configuration is more practical given recent advances in DL inference server designs. As the client, we use the NVIDIA Jetson Nano [42], a widely used embedded GPU platform for edge DL processing.

3.2. Server loads

First, we present the computational load that the EfficientNet-B5 model adds to the server when handling inference request queries. The main reason for performance degradation (latency increase) with increasing requests at the server is mostly due to the queuing delay (c.f., Fig. 1), since the server can only start processing requests when the current inference operation completes. Fig. 2(a) presents a breakdown of the inference completion time for a idle server for different model partitioning indexes (i.e., layers). Compared to this, Fig. 2(b) presents results when the server is currently handling 50 requests per second. Notice that the main difference between two is in the existence of added queuing latency (~100 ms). Thus, a busy server suggests that it may be beneficial to suppress model offloading and perform local inference operations. This indicates that comprehensively understanding the server operations, from inference processing latency to queuing latency is important when determining the fastest option of completing an inference task.

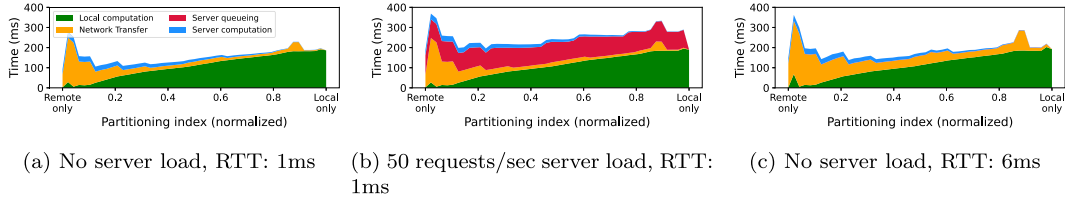


Fig. 2. Collaborative inference latency observed at different offloading (partitioning) points of the EfficientNet-B5 model with respect to different server load and communication link quality configurations.

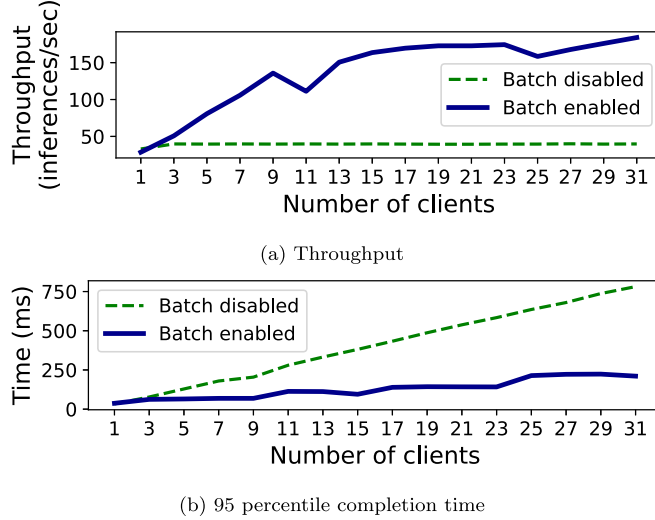


Fig. 3. Throughput and latency with and without batch inference capabilities at the server.

3.3. Network delays

Given that model offloading requires data transmissions, network condition changes can impact the overall latency; thus, affect model partitioning decisions. To quantify their impact, we manually induce latency on the communication link between the server and client devices using *tc-utility* [43]. Plots in Fig. 2(a) represent a scenario where the communication latency is minimal (e.g., 1 ms) and Fig. 2(c) presents a case where the latency was set to 5 ms. It is ideal to select the partitioning point with lowest overall latency; thus, with good wireless channel quality, the offloading point would be at the 0% mark (beginning) of the model (i.e., full model offloading), whereas with increased communication latency, offloading takes place at the 22% mark of the model. This simple experiment shows that changes in network conditions can heavily affect the client's model partitioning decisions and an accurate understanding of the network is needed to make better partitioning decisions.

3.4. Multi-batch inference

As our preliminary results show, network latency, and the server load are both important factors in model offloading decisions. As the last part of our preliminary studies, we examine the impact of applying batch inference operations on the queuing and inference latency.

Servers that support batch inferencing processes multiple inference requests concurrently to maximize the system-level throughput. On the other hand, with batch processing, the inference latency itself can lengthen with respect to the number of requests included in a single batch [19,21,22]. Fig. 3 presents the server throughput and 95th percentile inference completion time (with and without batch processing) for increasing number of clients issuing back-to-back requests to the server. The maximum throughput without batch processing saturates at 40 inference requests/second, while with batch processing, we see an increase to ~170 inferences/sec with 31 inference requesting clients. While the latency for each inference request may increase, as Fig. 3(b) shows, the overall task completion time is kept low (by more than two fold) when batch processing is enabled.

When given a target batch size, a server with batch processing capabilities will wait until the target number of instances fill up in the queue to initiate inference operations. As a result, the processing latency increases on a per-request perspective. On the other hand, the NVIDIA Triton server employs dynamic batching to address such inefficiency in which the wait time is bounded by a predefined duration. With less requests in the batch, the Triton server will start batch inference operations with available requests

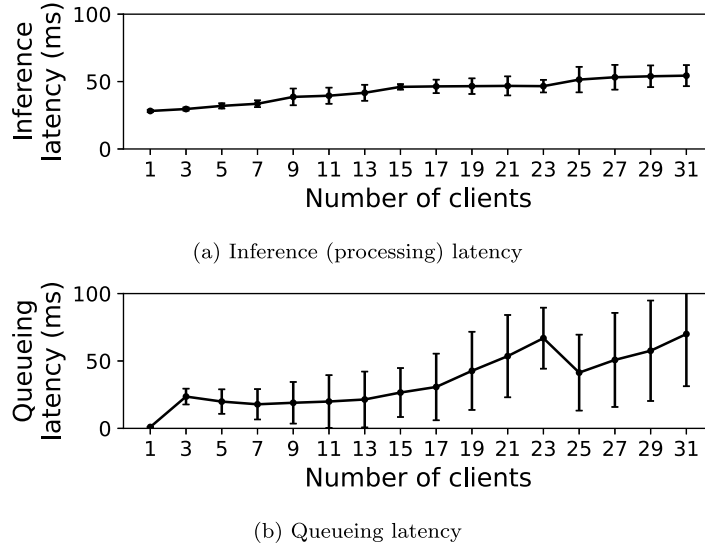


Fig. 4. Latency consumed for inference processing and queuing with increasing number of clients connected to the server (i.e., increased server load).

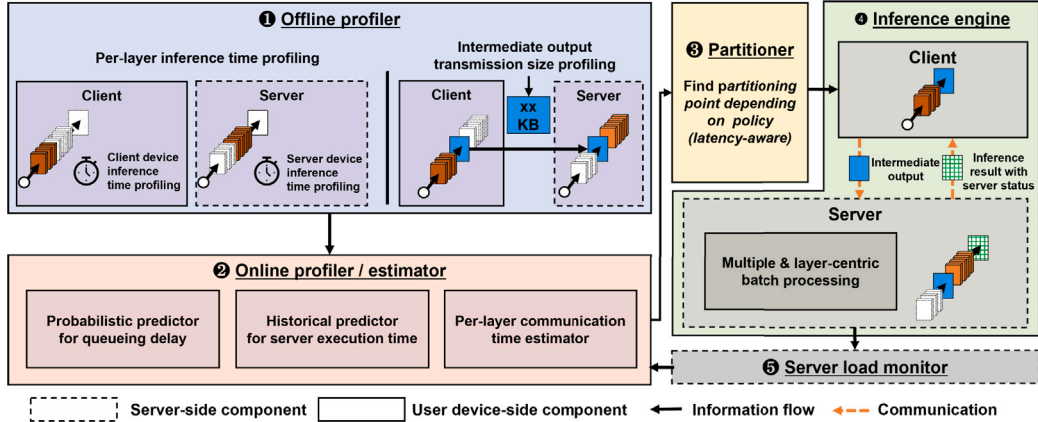


Fig. 5. Overview of DIAMOND's operational flow.

that consist the current batch. Such operational characteristics complicates the accurate estimation of server-side processing latency compared to simple non-batch server designs.

Fig. 4 presents the inference latency and queuing latency with increasing server loads. With increasing load, the inference latency grows gradually as more requests are included in the processing batch. Similarly, since higher server loads create queue-level back log, the queuing latency also gradually increases. Note that the queuing latency is measured from the request arrival time at the queue until it begins its initial inference processing. Therefore, if a request arrives just before the beginning of a new batch being processed (admitted in the batch), the queuing latency will be significantly lower than those arriving directly after the beginning of new batch, requiring them to wait for the current inference processing to complete. This suggests that high variations are expected in queuing latency measurements, and also implies that accurately measuring the queuing latency is a very challenging task. Nevertheless, without a precise estimate of this latency, accurate predictions on model offloading or the partitioning point is hard to achieve.

4. Diamond overview

We now provide a high-level overview of DIAMOND, our proposed end-to-end system for effective collaborative deep learning inference. Note that we consider a system architecture where multiple embedded clients connect to a server to either fully or partially offload inference operations with the objective of *minimizing the end-to-end inference latency* while controlling the latency impacting factors as described in Section 3. As Fig. 5 shows, DIAMOND consists of both offline and online operations, that make up

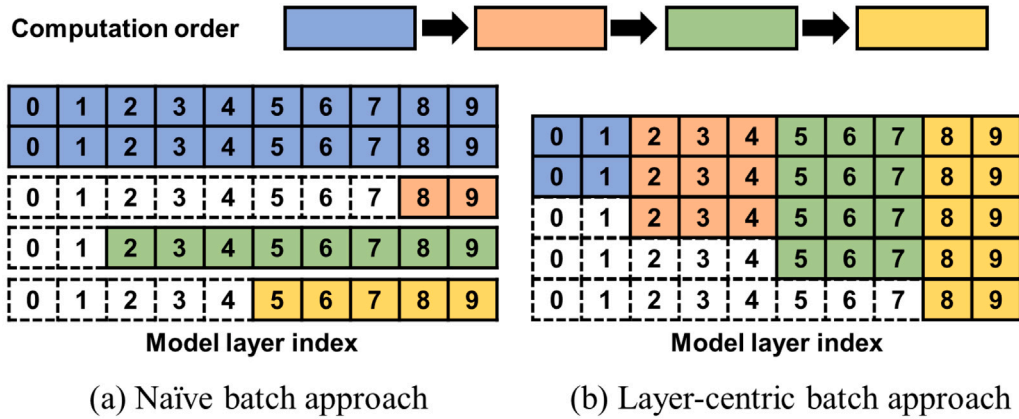


Fig. 6. Sample operation traces for five different incoming inference requests (at different split layers/points) with the naive and layer-centric batch processing approaches.

a lightweight and accurate system for latency-aware DL model offloading. Each number presented below indicate their illustration in Fig. 5.

❶ **Offline Profiler** For effective (and efficient) collaborative deep learning, it is important to well-understand the expected latency of inference processing both at the client and server. The offline profiler takes processing time measurements for the full and partitioned versions of the target model, and computes output sizes at each intermediate layers. These measurements are used as reference when predicting the processing latency. Note that the offline profiler executes once for each device-model configuration prior to deployments. Section 6.1 details the offline profiling phase.

❷ **Online Profiler/Estimator** In run-time, the online profiler makes measurements to understand the “current” conditions and performs *predictions* on the latency based on dynamic characteristics of the server and communication link. The server-side latency estimation, both processing and queuing latency, relies on recent performance history. Using this information *DIAMOND* computes the probability of completing an inference operation within the target latency. For communication link latency predictions, *DIAMOND* performs TCP-level simulations on the expected latency per-potential partitioning layer. We present details on these estimation operations in Section 6.2.

❸ **Model Partitioner** *DIAMOND* targets to identify partitioning points in a deep learning model to minimize the overall inference processing latency. Specifically, the model partitioner examines all possible partitioning points and determines the layer in which the overall latency is minimized. Using the online profiler results, the model partitioner examines the probability of completing inference task within the latency requirement for each target latency and comprehensively makes a decision for model partitioning.

Server-side component On the server side, *DIAMOND* consists of the ❹ batch-inference engine and ❺ server load monitor. The batch-inference engine executes inference requests in multiple batches by exploiting a novel layer-centric batch processing approach. Specifically, while employing the batch processing capability of the NVIDIA Triton server, given the heterogeneous request inputs that *DIAMOND* needs to support, we redesign the processing schedule on a per-partitioned-layer perspective rather than the request arrival time. This approach is to maximize server processing throughput. We present detailed operations in Section 5.

5. DIAMOND server design

In this section, we introduce *DIAMOND*'s inference server design which maximizes the inference throughput via a layer-centric batch processing approach. Specifically, layer-centric batch processing maximizes the advantage of applying batch-level parallelism for deep learning model layer partitioning. We also introduce a server load monitor which offers client devices with the current load status of the inference server in an on-demand manner by tracking the server-side computational latency.

5.1. Layer-centric batch processing

Given the benefits presented in Section 3.4, *DIAMOND*'s server employs a batch inferencing-based processing structure as a way to maximize its inference request processing throughput. With batched inferences, the server simultaneously processes multiple requests from the request queue via parallel processing. When doing so, given that a server GPU can load a single model instance at time, inputs for all parallel operations should be of same channel size, width, and height. Note when the server is used as part of a collaborative inference system, clients will process a portion of the model and issue heterogeneous request instances at different partitioning points. This means that the input characteristics will differ among requests in the queue, making it challenging to fully

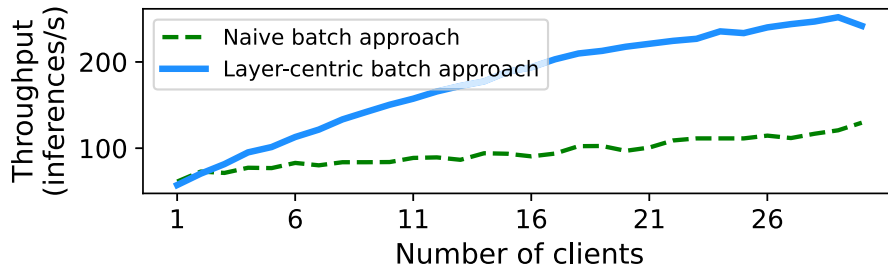


Fig. 7. Throughput performance comparison of the naive and layer-centric batch processing approaches.

exploit batch-level parallelism at the server. One intuitive approach is to group inference requests made for the same layer as a batch. However, this can be ineffective as model offloading points diverge and only a small number of overlapping inference requests are available. Such behavior will result in an operation schedule similar to a naive batch approach as we illustrate in Fig. 6(a). Here, five different inference requests, each offloading at layer 0, 0, 8, 2 and 5, arrive at the server. Using a naive approach, a total number of four separate inference operations are required to complete the task in the order presented at the top of the figure. Unfortunately, not too much performance gain from exploiting parallelism can be expected in such scenarios.

To fully exploit server parallelism and maximize the throughput even under such scenarios, *DIAMOND* employs a novel **layer-centric batch processing** approach. Specifically layer-centric batch processing allows for concurrent inference processing regardless of their requested partitioning points. As we illustrate in Fig. 6(b), in the layer-centric batch processing approach, all queued requests are sorted based on their requested partitioning point. Instead of per-request processing, layer-centric batch processing processes requests on a layer-wise dimension, in which overlapping model layers are grouped and batch-processed.

In Fig. 7 we present the throughput of the naive and layer-centric batch processing approaches for varying number of inference requesting devices (i.e., clients). The Triton server is configured to support collaborative inference for the EfficientNet-B5 model and clients send back-to-back inference requests at random partitioning points. We can notice from the results that layer-centric batch processing improves the throughput by >2× compared to the naive approach by fully exploiting server-side parallelism.

5.2. Server load monitor

DIAMOND implements a server load monitor as the communication end-point between clients and the server. The server load monitor keeps track of historical performance data which includes inference processing and queuing latency trends. As we detail in Section 6.2.2 *DIAMOND* exploits a probabilistic model for accurate server-side latency predictions at different potential model offloading points.

Given that model partitioning decisions take place at the client and latency predictions are made at the server, it is important to carefully design how server-side latency information will be delivered between the two. In scenarios where frequent inference requests and responses are exchanged, the server can piggyback latency measurements on response packets to minimize communication overhead. However, when the client performs local execution, due to the lack of packet exchange, the client will not be notified even when the server conditions improve. For this reason, in *DIAMOND*, the server load monitor is a virtual process, which upon request from the client, replies with a summary of the expected server-side processing latency. The request packet is designed to be small, allowing for a light-weight operation compared to exchanging full-size inference requests as done in most previous work [13].

6. Profiling and latency prediction

We now introduce *DIAMOND*'s latency profiling and prediction mechanisms for the offline and online phases using the following sections.

6.1. Offline profiling

DIAMOND, in its offline phase measures device-specific DNN model execution times at both the client and server platforms and computes model-specific offload data sizes (at different potential offloading layers). The processes discussed in this section are performed only once prior to system deployment, and the profiled information is stored on the client to use when identifying a potential partitioning point together with the knowledge gained in the online profiling phase.

6.1.1. Client/server inference latency profiling

Accurate latency profiling requires a clear understanding of two processing components. First is the latency at the client for model processing. For this, we measure and collect the time a client spends for processing a DNN model inference operation at each layer of a deep learning model, $C_i^{offline}$, $i \in L$, where L is the set of all layers in a DL model. Next, we perform the same profiling operations on the server for each potential offloading point $S_i^{offline}$. Note that this is a one-time process that can be performed offline prior to the actual operations. We also point out that since modern inference servers improve their performance via batch processing, measurements for $S_i^{offline}$ is only used when the inference server is idle, as baseline measurements. When the server is not idle, baseline measurements are not used and we will discuss in Section 6.2.2 on how latency predictions are made with varying server load.

6.1.2. Offload data size profiling

An offloading decision made at different offloading points not only result in different server-side computational burden, but also induce varying amounts of data/parameters to be exchanged over the network. Naturally, more transmission data results in longer latency, and less data will be transmitted faster. In the offline profiling phase, *DIAMOND* analyzes the model's potential offloading points to determine how much data exchange each point would require. Given a DL model, this is a straightforward process and *DIAMOND* maintains a table with the potential offloading points as the key and the amount of data to be transmitted as the value. Later, as we will discuss in Section 6.2.1, this table is used to determine the final offloading point by translating the data size to expected network latency with respect to the up-to-date network conditions.

6.2. Online profiling and estimations

The second phase of *DIAMOND* is dedicated to computing the expected latency at the server and network given the current conditions in run-time, and finally determining the ideal offloading point that minimizes inference latency. We note that online profiling capture measurements on factors that impact the communication and server-side latency. For example, these include factors such as the round trip time between the server and clients and the historical analysis of queueing delay and server processing latency. Once the profiling is complete, *DIAMOND* clients make estimations on the communication delay and server-side latency according to the collected (profiled) measurements.

6.2.1. TCP simulation-based data communication latency estimation

The wireless communication latency is a critical factor determining whether or not computation offloading is beneficial on an end-to-end latency perspective. To accurately estimate the communication performance, previous works exploit iPerf3 [23] or similar tools to measure the available network bitrate [7,11,26]. Unfortunately these schemes induces significant network/energy overhead, making them less suitable for real-time model offloading scenarios [44].

To alleviate such overhead, some recent work assess network status by measuring the latency experienced by previous model offloading packets [13]. Specifically, the measured latency is divided by the packet size to estimate the link's bitrate. While simple, there are two major limitations. First, we note that offloaded data sizes are typically <1000 kB, and that data exchange typically uses TCP. Since TCP exploits a slow start mechanism, small size data exchange completes before reaching the maximum bitrate. Thus, with estimations made under network conditions where the TCP congestion window plays a critical role, different data sizes can lead to inaccurate predictions [29,30]. In other words, accurate network latency predictions should consider the non-linearity of TCP links.

The second limitation is that for the network bitrate estimation to be fully implicit, inference requests should be made back-to-back. Since mobile networks show dynamic behavior, latency information collected from past data exchange, may no longer represent current network conditions.

DIAMOND acknowledges the benefits of exploiting information from already-existing packet exchanges. However, *DIAMOND* exploits the fact that data communication latency should be TCP operation-aware, and that only recent latency measurement samples should be used for the estimations. Note that when an inference initiates at the client, a new HTTP connection with the server via 3-way handshaking is established. For this, a packet is sent to the server and the client can *profile* its round trip time (RTT) to capture the experienced network latency [45]. Based on such initial measurements, the key idea of our approach is to “simulate” a TCP operation for the next data to be sent. Specifically, the latency measurements from the HTTP handshaking packet is used to adjust the parameters for this TCP simulation (e.g., window size changing dynamics, time required to complete window transmission, etc.). Using the maximum segment size (MSS) unit, we compute how many TCP windows the next packet will consume while considering the TCP operations. When offloading computation, the online profiler is provided with updated parameters for accurate TCP simulation. As long as continuous offloading operations take place, no further packets are explicitly sent, and results from recent offloading packets are used. If the offload operations discontinue for a preset time limit, the HTTP handshaking process is repeated so that the up-to-date network conditions are reflected to the network latency estimation process.

We finally mention that the use of TCP can be counter-intuitive for systems with energy constraints as considered in this work. However, in model offloading scenarios, the reliability of packet transmissions from the client to the server (and vice-versa) is of utmost importance as missing data due to the use of other (less reliable) protocols can lead to malfunctioning inference operations at the remote server. For this, many previous model offloading work focus on using TCP as the networking layer protocol. We too take the observations from such previous work and exploit TCP as the main networking mechanism for *DIAMOND*.

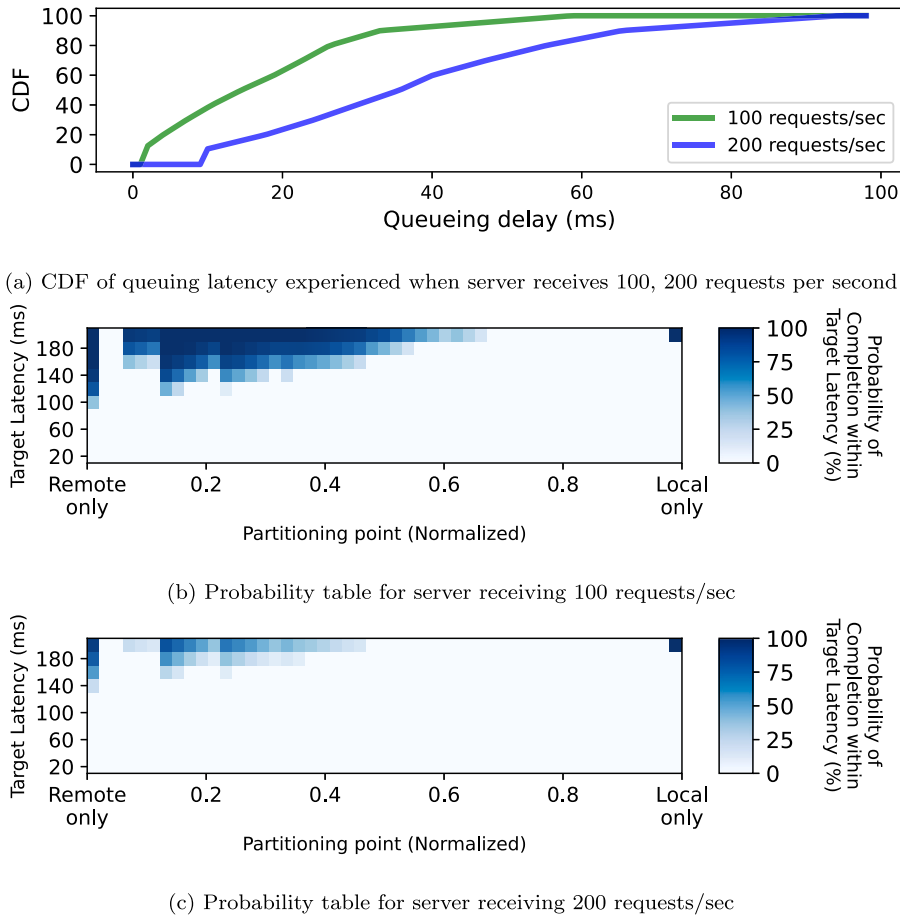


Fig. 8. (a) Queuing latency experienced for different server loads and (b, c) probability table for different target latency with different server loads at varying partitioning points of the EfficientNet-B5 model.

6.2.2. Server inference processing latency prediction

The server inference time is dependent on the number of jobs present in the batch request queue and the requested partitioning point of the model for each queued job. If there exists only a single request in the queue, the server inference time equals the offline phase measurement, $S_i^{offline}$. With heterogeneous requests in the server queue and using the layer-centric batch processing approach, as discussed in Section 5.1, the server inference time will show a much more diverse pattern; thus, an effective on-line prediction of the server latency is needed.

DIAMOND presents different behaviors in predicting the server inference time with respect to server load conditions. When a client's inference request arrives while the server is idle, the predicted server offline-computed latency. On the other hand, with queued requests, DIAMOND server maintains a moving average of the previous server processing latency samples. We make such a decision based on the observation that the server load does not show instantaneous changes, rather gradual increasing/decreasing behavior.

As discussed in the preliminary studies, compared to server inference latency, the queuing latency shows higher variability [46]. An inference request will be served immediately if the server is idle, but in other cases, they will experience some level of queuing latency. Given that this delay is added to the overall latency, it is important to accurately model (and predict) the queuing latency prior to partitioning decisions. In DIAMOND we take a probabilistic approach in predicting the queuing delay of incoming inference requests. Specifically, the DIAMOND client computes the probability of a round trip response from the server for different target (expected) delays by combining the local processing latency (C_i), communication latency (T_i), and queuing latency (Q), and server processing latency (S_{online}). Using the precomputed C_i , T_i and S_{online} , we can compute the acceptable Q range. Finally, DIAMOND's client can profiles the server queue status using the queuing latency history logs (distribution) offered at the server and constructs a *probability table*, which presents the probability of completing the inference within a target latency.

Fig. 8(a) shows the cumulative distribution function (CDF) of queuing latency when server receives 100 and 200 requests per second. We can see that the median values of both cases are 16 ms and 37 ms, respectively. This means that when the server receives more requests, the acceptable Q range becomes tight and it is more challenging to satisfy the given target latency. We visualize sample probability tables in Fig. 8(b) and (c) for a server with different loads, with respect to queuing delays observed in Fig. 8(a).

Algorithm 1 Latency-Aware Policy

```

1: Input:
2:  $N$  : number of layers in the target DNN model
3:  $prob\_table$  : probability table
4:  $prob\_thresh$  : probability threshold
5: Output: Partitioning point  $p$ 
6: procedure GETLAPoint
7:   for  $t \leftarrow 0$  to  $length(target\_times)$  do
8:      $points \leftarrow [], probs \leftarrow []$ 
9:     for  $i \leftarrow 0$  to  $N$  do
10:      if  $prob\_table[i][t] \geq prob\_thresh$  then
11:        Add  $i$  to  $points$ 
12:        Add  $prob\_table[i][t]$  to  $probs$ 
13:      if  $points$  is not empty then break
14:       $p \leftarrow points[argmax(probs)]$ 
15:   return  $p$ 

```

7. Run-time partitioning decisions

Using the latency prediction probability discussed in the previous section, *DIAMOND* targets to offload model computations from the client at a point where the overall inference latency can be minimized. Algorithm 1 presents the *DIAMOND* operations when applying latency-aware partitioning. $prob_table$ is the probability table as discussed in Section 6.2.2 representing the probability of inference completion within a target duration at each partitioning point. $prob_thresh$ is a predefined threshold indicating the minimum probability for partitioning point selection. Specifically, with latency-aware partitioning, clients will iterate through Algorithm 1 by starting with a low completion time target and gradually increasing the target latency until a partition point with a high-enough probability is found. If multiple points meet the target latency, *DIAMOND* selects the partitioning point with higher probability.

8. Evaluation

We present our evaluation configuration details and results for validating the performance of *DIAMOND* using the following sections.

8.1. Experimental setup

For evaluations, we use a full implementation of *DIAMOND* operating on the NVIDIA Jetson Nano (as the client) and the NVIDIA Triton server as in Section 3. We take the EfficientNet-B5 architecture as the target deep learning model. Furthermore, we take other versions of the EfficientNet architecture, B4 and B6, along with ResNet-101, ResNet-152 and VGG-16 so that we cover models of various complexities and layer characteristics.

For comparisons, we take the following cases. (i) a remote only scenario where all inferences are immediately offloaded, and (ii) the SPINN [13] approach. Specifically, for SPINN, we compare the network and server load estimation sub-components in greater detail. Note that SPINN estimates network conditions by dividing empirical latency measurements with the transmitted data size as discussed in Section 6.2.1 and the server load prediction relies on a scaling factor (ratio between $S_t^{offline}$ from the offline phase and the reported server processing latency S).

For our evaluations, we connect clients to a WiFi network and an externally connected load generator issues additional inference requests to the server based on a Poisson distribution-based arrival time. To vary the RTT between the client and server, we use the $tc_utility$ as we aforementioned in Section 3.3 [43]. We note that, the latency is measured as the end-to-end time from when the client device issues an inference request to the server until an inference response is received and is measured on each client device. For energy overhead measurements, especially for the wireless component, due to the lack of an accurate energy measurement platform on a per-component basis, we exploit the formulations made by Huang et al. on WiFi energy consumption [47].

8.2. Performance comparison with naive approach

We compare *DIAMOND* to the remote-only approach in terms of server throughput improvement and completion time improvement on the client side. Fig. 9(a) plots the maximum throughput level with an external load generator issuing back-to-back inference requests to the server at different points in the EfficientNet-B5 model (e.g., 0% — all inference operations take place at the server, 22% mark, 33% mark, and random points). When the server continuously receives full inference requests (e.g., 0% — remote only approach), the throughput supported at the server saturates at ~160 inference operations per second. Whereas, with loosened request loads (e.g., 22% and 33% cases), we can observe an increased throughput of 428.8 and 520.4 inference operations per second, respectively. For a more realistic scenario where requests at random points of the model are requested, we observed a

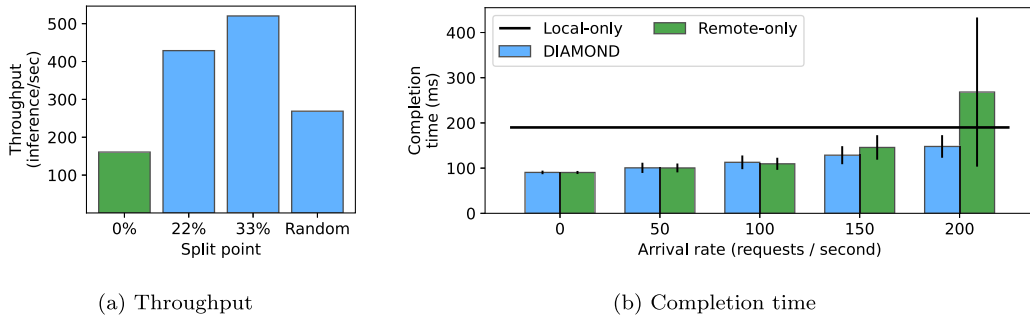


Fig. 9. (a) Server throughput comparison between remote-only and partitioning approaches, (b) average request completion time among remote-only, local-only and *DIAMOND* with respect to server load status (Model: EfficientNet-B5).

throughput of 268.6 inference request per second. This suggests that it is meaningful (in terms of maximizing server-side inference processing throughput) to identify effective model offloading points rather than offloading all inferences to the server, which allows a single server instance to support more clients.

In Fig. 9(b) we present the completion time for the remote-only approach and *DIAMOND* with respect to varying server loads. Here, we configure the load generator to issue full model inference requests for the remote-only case and random point requests for *DIAMOND*. With arrival rates exceeding 150 requests per second, *DIAMOND* clients complete inferences faster than the remote-only approach. Moreover, when the arrival rate reaches 200 requests per second, a remote-only approach can be slower than a local-only approach (i.e., no offloading) due to server congestion.

8.3. Impact of server load dynamics

Next we examine the performance of different offloading approaches with different server load dynamics. In Fig. 10(a) we present a dynamic server load pattern used for our experiments. We selected this continuously increasing pattern, among many, given that such a behavior will be common for IoT-cloud collaborative inferencing systems where an increasing number of edge devices are added to the system as the system (or application) scales. Fig. 10(b) shows the completion time for this varying server load pattern, and Fig. 10(c) and (d) show the model offloading points (i.e., split points) for *DIAMOND* and the baseline in response to the dynamic server load. Here, we observe that *DIAMOND* takes a conservative approach in partial offloading, instead, it fully offloads inference operations until the arrival rate reaches a reasonable point (i.e., 280 requests per second), whereas the baseline performs partial offloading from 150 inferences per second. With 150 requests per second of server load, the completion time difference between the baseline approach's decision and *DIAMOND* is approximately 30 ms, which leads to 30% latency reduction when using *DIAMOND*. The reason for the baseline's performance is because the slowdown factor used for server load estimations aggressively increases even under light server loads.

Fig. 11(a) depicts the average and standard deviation for the completion time (1000 runs) under varying server loads. Plots indicate that there is no significant difference between the two approaches until the server processes 100 requests per second given that both fully offload the model until this point. However, the baseline shows prolonged completion times when the arrival rate exceeds 150 requests per second, similar to our observation in Fig. 10. At 250 requests per second, both show (again) similar performance as both approaches decide to perform most operations locally.

Fig. 11(b) presents the packet (left) and energy (right) overhead for the same configuration when issuing 1000 inference requests. Note that the baseline approach updates its scaling factor by issuing dummy inference requests until the server load status is considered suitable to send a request (i.e., less latency compared to local execution). On the other hand, *DIAMOND* uses its server load monitor (Section 5.2) to share and identify server load status on an on-demand basis to the client device, which suppresses the need for explicit dummy requests. Thus, *DIAMOND* shows no explicit requests while the baseline sends approximately 770 requests (due to frequent local processing and lack of current server status data) with a server load of 250 requests per second. Specifically, the baseline's dummy request sends requires 588 KB (i.e., size of input data), whereas *DIAMOND* transmits one MSS-sized packet (1448 bytes). Based Huang et al. [47] sending 588 KB through WiFi on mobile devices (e.g., HTC mobile device used in their experiments) results in 1.37 J of energy usage while sending 1448 B requires only 3 mJ. With 250 requests per second load, a baseline client spends 1 kJ, compared to 10 J for *DIAMOND* when refreshing server load information. We note that 1 kJ overhead is the amount of energy to execute the EfficientNet-B5 model on a local device 1k times, which is enough to have a negative impact on system lifetime.

Thus, despite similar latency performance for the 250 requests per second server load scenario in Fig. 11(a), results indicate that *DIAMOND* is more efficient.

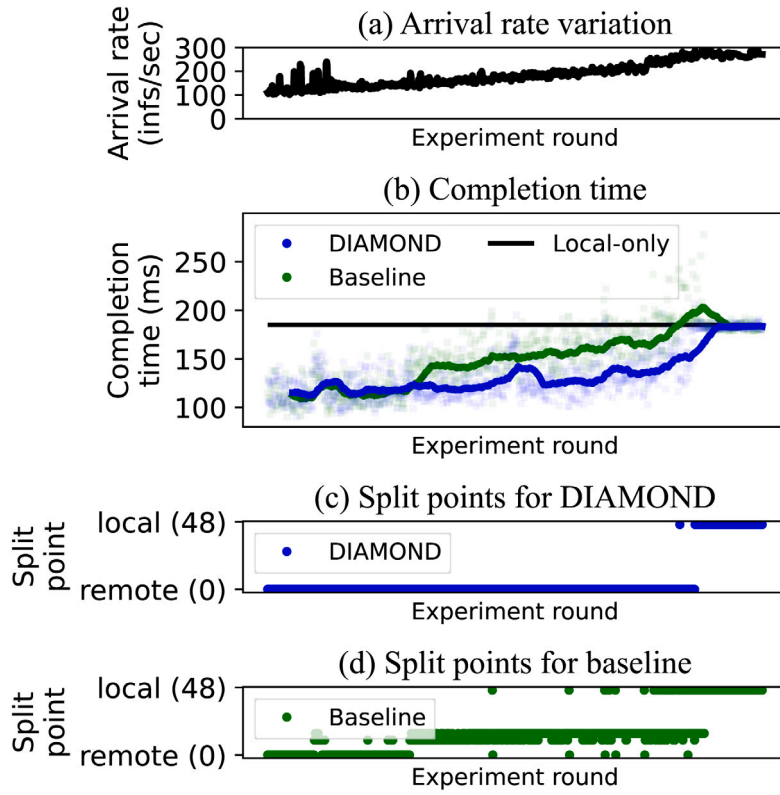


Fig. 10. (a) Arrival rate variation used in experiment to differ server loads, (b) resulting completion time for *DIAMOND*/local-only/baseline, (c) split points (i.e., offloading decision layer) for *DIAMOND*, (d) split points for baseline. The x-axis indicates the different experiment cases with different arrival rates (independent experiment rounds). (Model: EfficientNet-B5, probability threshold: 80%).

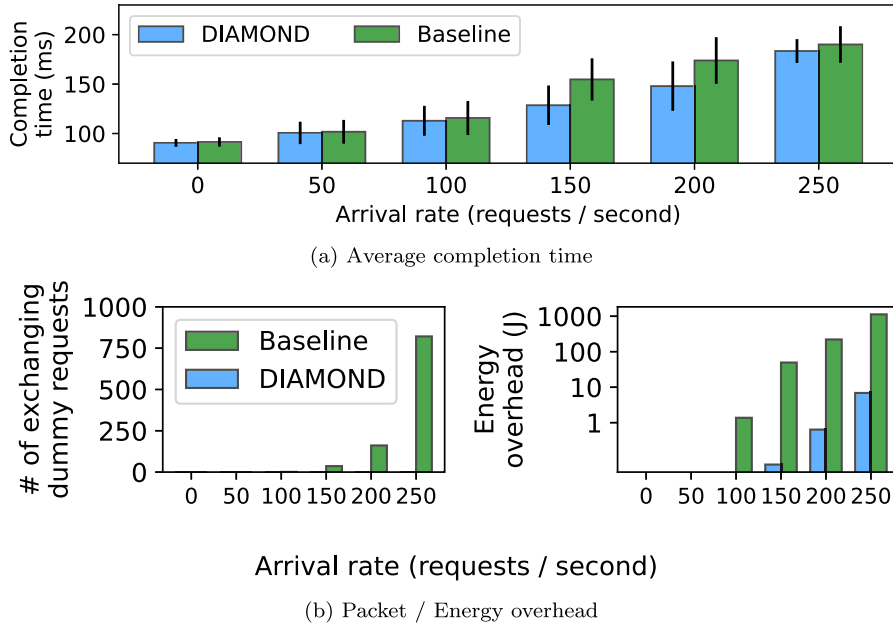


Fig. 11. (a) Average completion time under varying server load, (b) number of exchanged dummy requests and energy overhead for capturing current server status at the client (Model: EfficientNet-B5, probability threshold: 80%).

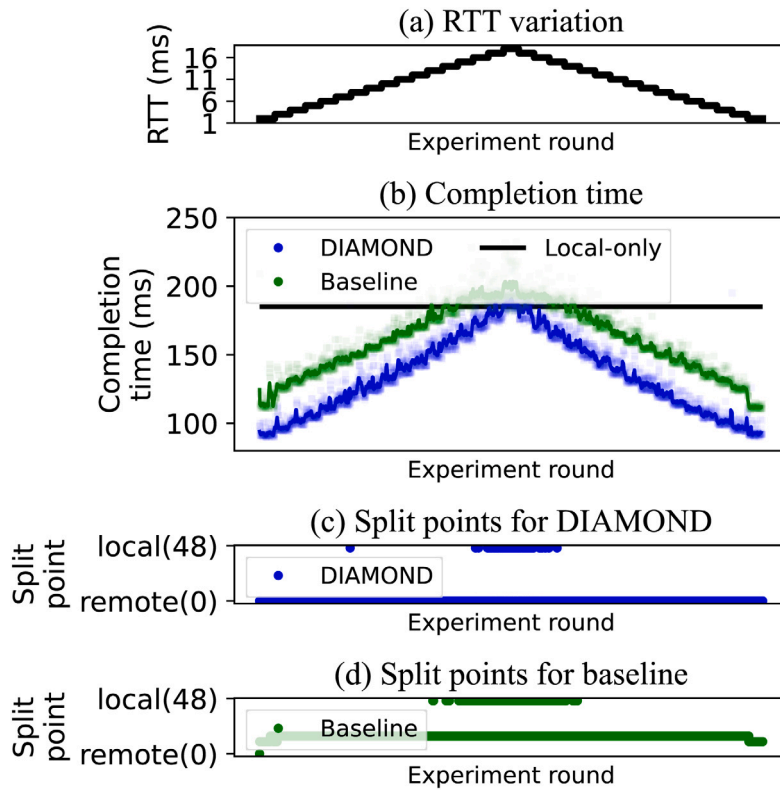


Fig. 12. (a) RTT variation used in experiment to differ link conditions, (b) resulting completion time for *DIAMOND*/local-only/baseline, (c) split points (i.e., offloading decision layer) for *DIAMOND*, (d) split points for baseline. The x-axis indicates the different experiment cases with different RTTs (independent experiment rounds). (Model: EfficientNet-B5, probability threshold: 80%).

8.4. Impact of communication quality dynamics

Next we evaluate *DIAMOND* and the baseline approach with varying communication link qualities between the client and the server. As discussed in Section 6.2.1, *DIAMOND* exploits a novel TCP-simulation-based latency prediction scheme, while the baseline relies on direct measurements from previous model offloading attempts. Fig. 12(a) plots the varying RTT used in our experiments to differ the network/communication conditions (gradually increasing RTT from 1 to 18 ms and back down) and Fig. 12(b) plots the experienced inference request completion time for *DIAMOND* and baseline, and Fig. 12(c) and (d) show that the model split points for the two approaches, respectively. Again, *DIAMOND* shows a conservative behavior in partially offloading the model, and issues full model offloading requests until the RTT reaches ~ 13 ms. Furthermore, we can notice that once *DIAMOND* identifies harsh network conditions, it quickly adapts to a fully local operation and does not try partial offloading approaches under such conditions. Thus, the worst-case latency for *DIAMOND* does not exceed the full local operation case.

When comparing the split points for the two cases, the baseline's performance shows a noticeably different pattern by aggressively trying to partially offload the model (mostly at layer 16) due to three major reasons. First, the baseline performs partial offloading at split point 11 despite only a small degradation in communication performance. A worse communication quality leads to splits taking place at split point 16. While partial offloading leads to shorter communication latency (compared to fully offloading the model), it increases the processing time at the (slower) local platform. As a result, the overall latency is higher compared to the fully offloading approach taken by *DIAMOND* under the same conditions. Second, note that the baseline approach relies on the transmission of a dummy request packet to analyze the current wireless channel conditions. Here, given the discrepancy between the dummy request packet size and actual offloading data, an accurate prediction of the transmission latency is difficult. When the communication quality is extremely poor (e.g., high RTT), the baseline shows unstable performance. This can be explained by the fact that faulty network latency prediction can lead to model offloading, only to realize that the network conditions are poor; thus, the next inference operation is performed locally. Repeatedly, faulty predictions occur again, resulting in unstable offloading decision making even when the network is continuously slow. Finally, even when the network quality gradually improves, the baseline approach does not select to fully offload the model since its network quality estimations cannot quickly adapt to the improved conditions.

Fig. 13(a) shows the average completion time and standard deviation for *DIAMOND* and the baseline approach under varying RTTs (i.e., network conditions). Specifically, we observe 100 measurements for each RTT status while increasing the RTT from 1 to

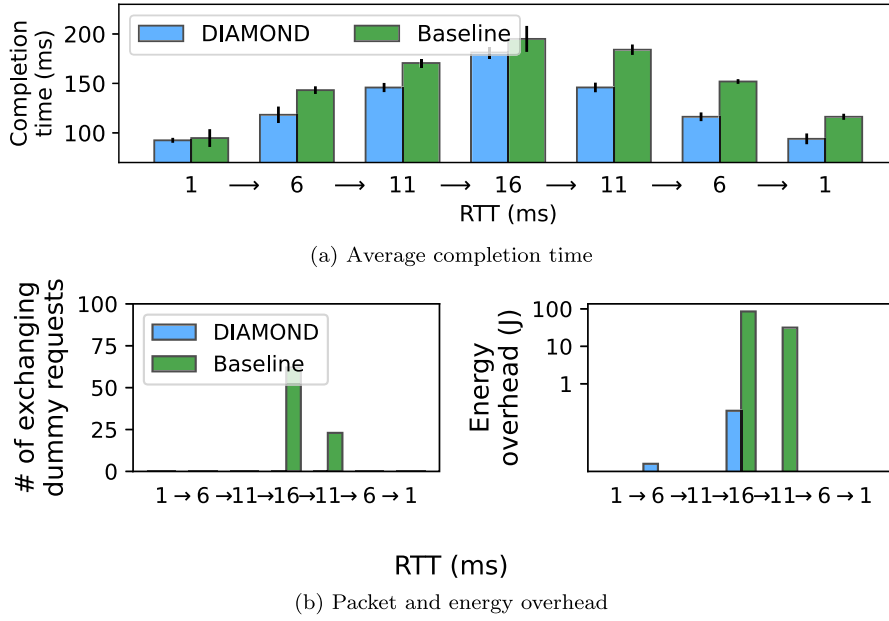


Fig. 13. (a) Average completion time with varying communication link quality, (b) number of exchanged dummy requests and energy overhead for capturing current server status (Model: EfficientNet-B5, probability threshold: 80%).

16 ms, then decrease the RTT from 16 to 1 ms and present the measurements at intervals of 5 ms in Fig. 13. For the initial 1 ms RTT case at the beginning, the completion time is similar for the two approaches. However, when RTT increases to 6 and 11 ms, *DIAMOND* takes 118.27 ms and 145.85 ms, respectively, while the baseline takes 143.2 ms and 170.61 ms, respectively. As we observed in Fig. 12, this is because the baseline approach performs partial offloading at an early stage due to naive communication status profiling. When the RTT reaches 16 ms, *DIAMOND* either fully offload the model or perform local-only execution, while the baseline decides to either partially offload its computation or perform local execution. This results in the longer completion time for the baseline (195.02 ms), which is in fact even longer than fully executing the model locally (183 ms). On the other hand, we see that *DIAMOND* successfully manages to minimize the latency to 181.23 ms for the same scenario.

Interestingly, when the RTT reduces back to 1 ms, the average completion time of *DIAMOND* is 93.95 ms (full offload), while the baseline still takes 116.25 ms. Here, we can notice that the baseline client performs partial offloading. This is due to the fact that the its network profiler cannot properly assess the improved network conditions; thus, does not consider full offloading as an option. Fig. 13(b) plots the packet and energy overhead for different RTT configurations. When the RTT reaches 16 ms, the baseline decides to perform local execution 62 times out of the 100 tested cases, causing 85 J of energy overhead, whereas *DIAMOND* decides to locally execute its operations only 26 times, leading to an energy overhead of only 78 mJ.

8.5. Impact of probability threshold

Our evaluations until now assumed a probability threshold (`prob_thresh`) of 80%. We now vary this threshold and examine how offloading decisions change for values of 70% and 90%. Specifically, we plot the split points and completion times for each case (with 80% threshold case) while varying arrival rates and RTT in Figs. 14 and 15.

When experiencing higher server-side loads, `prob_thresh` results in a more conservative model offloading decision. Specifically, Fig. 14 shows that *DIAMOND* with a threshold of 70% performs remote only operations promptly when the server is found busy compared to higher thresholds. On the other hand, *DIAMOND* aggressively processes inference requests locally when the probability threshold is high. Similar, but more prominent, observations can be made for RTT. As Fig. 15 shows, with a lower threshold value, more remote operations take place. Whereas, a higher threshold, introduces more local inference operations.

We note that `prob_thresh` can be configured by system developers with respect to the application goals. If minimizing local computational overhead is of priority, a high threshold can be set, while an application sensitive to latency will configure a lower threshold.

8.6. Serving various DNN architectures

Lastly, we examine the completion time for an inference request when using *DIAMOND* and the baseline approach with different deep neural network models (e.g., EfficientNet-B4, EfficientNet-B6, ResNet-101, ResNet-152, and VGG-16) with varying server load (in units of inference arrival rate) and network quality (continuous RTT from 1 to 16 then back to 1 ms) in Fig. 16. Here, we compute

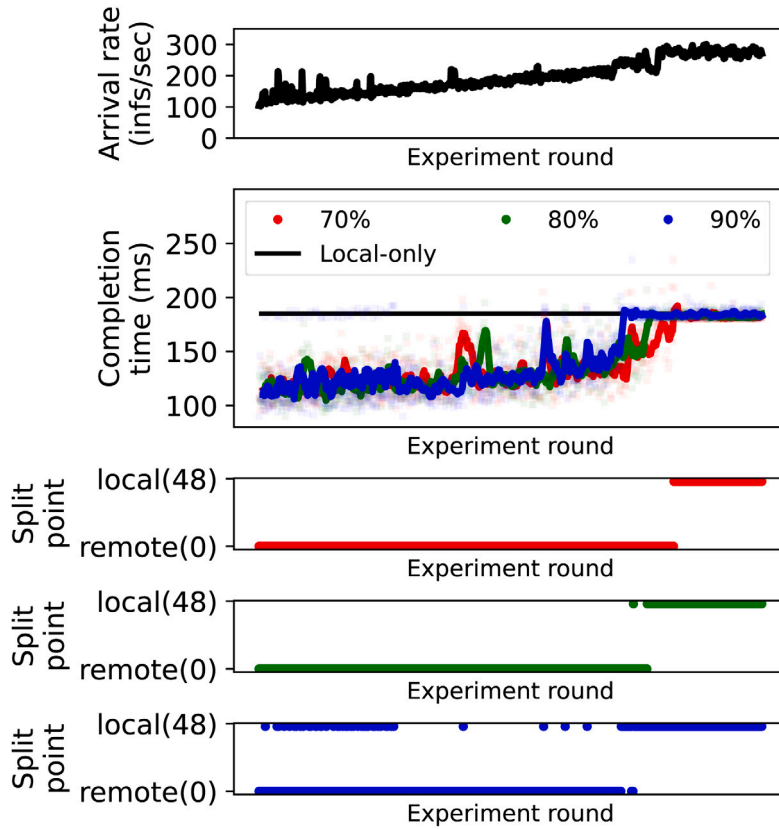


Fig. 14. Arrival rate variation used in experiment (top) and completion time for varying probability thresholds (second). We also plot model split points (i.e., offloading layer) for probability thresholds of 70, 80, and 90%, respectively.

the average inference completion time for 1000 inference requests. The result agree that *DIAMOND* contributes to reducing inference completion time under various models and conditions.

9. Discussions and future research directions

• **Supporting diverse communication environments:** *DIAMOND* exploits a TCP simulation-based network profiling tool for estimating the communication latency. We believe we can initiate two interesting discussions to improve the completeness of *DIAMOND*. First, we point out that this work focuses on the RTT as the most important networking latency estimation aspect, which reflects the communication quality. However, in reality, other factors such as packet retransmissions can also affect the networking latency. Second, while *DIAMOND* currently focus on TCP communications, given that its lossless networking capability is desirable in many applications, some may prefer other networking protocols. We note that as long as the bitrate can be computed, which for UDP for example is a simple process, all other operations of *DIAMOND* can sustain. Nevertheless, we identify the support for various network quality estimation factors and networking protocols as interesting direction of future research.

• **Considering client energy consumption:** Since mobile or wireless embedded devices have limited energy resources, it is important to increase system lifetime by reducing energy consumption at the software. To design an energy-aware partitioning policy, we need an accurate energy usage profile on a component level or need some way of estimating the energy consumption of model execution and communication for clients with respect to different partitioning points. Combining energy consumption information with *DIAMOND*'s run-time partitioning decision algorithm can help identify an appropriate partitioning point that balances inference latency and energy consumption, but we leave the validation of such approach as future work.

• **Further optimizations with orthogonal approaches:** *DIAMOND* focuses on improving the performance of collaborative inferences while considering recent (dedicated) deep learning server characteristics and IoT-specific communication characteristics. Prior works related to collaborative inference suggest schemes such as early-exit to reduce both computation and communication cost depending on server and communication status [13] and data compression using auto-encoder which reduces the size of data to be transmitted [12]. We note that *DIAMOND* is in fact orthogonal with these efforts and their benefits can easily be combined to offer an optimized solution. While interesting, such integration is out of the scope of this work and we leave this as a direction of future research.

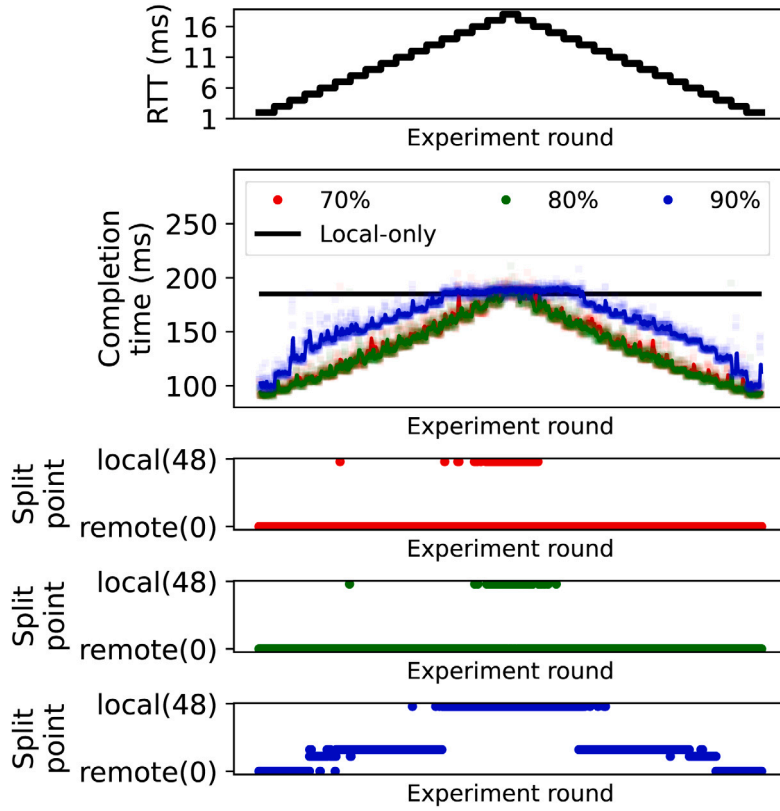


Fig. 15. RTT variation used in experiment (top) and resulting completion time for varying probability thresholds (second). We also plot model split points (i.e., offloading layer) for probability thresholds of 70, 80, and 90%, respectively.

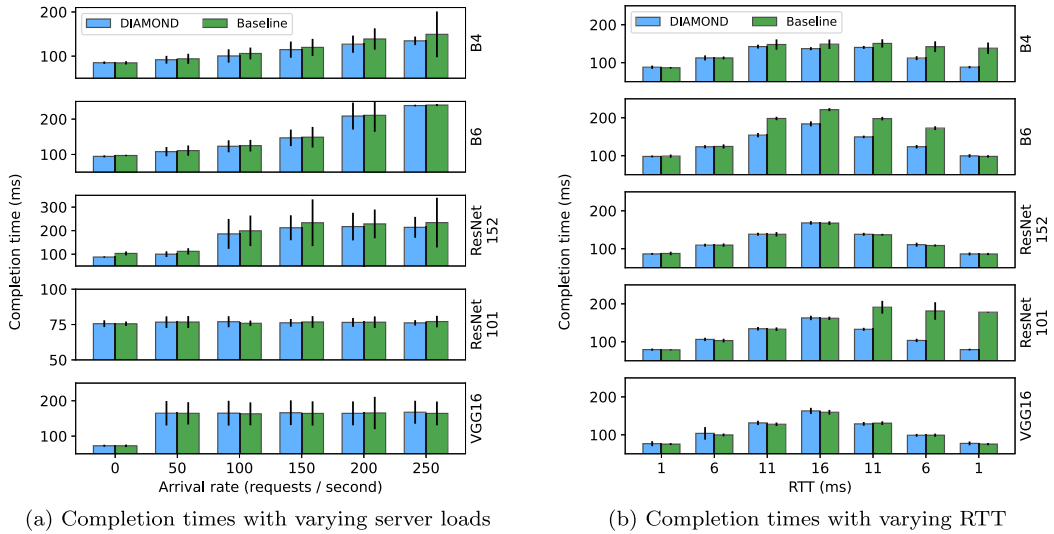


Fig. 16. Completion time performance for different models with varying server loads and RTT.

10. Conclusion

Computation offloading is an attractive technique when integrating deep learning model-based inference services on resource-limited edge platforms. Despite its potential, previous work possess limitations in accurately estimating the network latency and server load under practical conditions. In this work, we presented *DIAMOND* as a collaborative deep learning model offloading

technique that captures accurate latency estimations for state-of-the-art deep learning server architectures and varying network conditions. Specifically, *DIAMOND* combines a TCP-level emulation-based network estimation component and a batch processing-aware server processing latency prediction component to support each deep learning inference requests with accurate up-to-date latency estimations. We evaluate *DIAMOND* using implementations on an NVIDIA Jetson Nano and the Triton server. Results from comprehensive evaluations indicate that *DIAMOND* identifies model partitioning points that offering minimal latency operations with less network and energy overhead compared to previous approaches, and it well-adapts to dynamic network and server conditions.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: JeongGil Ko reports financial support was provided by Korea Ministry of Science and ICT. JeongGil Ko reports financial support was provided by IITP. JeongGil Ko and Jeongseob Ahn reports financial support was provided by ITRC Program. JeongGil Ko reports financial support was provided by Korea Ministry of Culture Sports and Tourism and Korea Creative Content Agency. JeongGil Ko reports financial support was provided by Ministry of Trade, Industry and Energy and KICT through the International Cooperative R&D program.

Data availability

Data will be made available on request.

Acknowledgments

This work was supported by the Ministry of Science and ICT's NRF Basic Science Research Program (2021R1A2C4002380), IITP (IITP-2022-0-00240), ITRC Program supervised by IITP (IITP-2021-2020-0-01461 and IITP-2021-2021-0-02051), by the Ministry of Culture, Sports and Tourism and Korea Creative Content Agency (R2021040018), and by the Ministry of Trade, Industry and Energy and KIAT through the International Cooperative R&D program under Grant (P0016150).

References

- [1] Microsoft, Azure machine learning, <https://azure.microsoft.com/en-us/free/machine-learning/>.
- [2] Amazon, Sagemaker, <https://aws.amazon.com/ko/sagemaker/>.
- [3] Google, Vertex ai, <https://cloud.google.com/vertex-ai>.
- [4] N.P. Jouppi, et al., In-datacenter performance analysis of a tensor processing unit, *SIGARCH Comput. Archit. News* 45 (2) (2017) 1–12, <http://dx.doi.org/10.1145/3140659.3080246>.
- [5] Qualcomm, Snapdragon 865 5 g mobile platform, <https://www.qualcomm.com/products/snapdragon-865-5g-mobile-platform>.
- [6] NVIDIA, Nvidia turing gpu architecture, <https://www.nvidia.com/content/dam/en-zz/Solutions/design-visualization/technologies/turing-architecture/NVIDIA-Turing-Architecture-Whitepaper.pdf>.
- [7] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, L. Tang, Neurosurgeon: Collaborative intelligence between the cloud and mobile edge, *ACM SIGARCH Comput. Archit. News* 45 (1) (2017) 615–629.
- [8] R. Hadidi, J. Cao, M. Woodward, M.S. Ryoo, H. Kim, Distributed perception by collaborative robots, *IEEE Robot. Autom. Lett.* 3 (4) (2018) 3709–3716.
- [9] A.E. Eshratifar, M.S. Abrishami, M. Pedram, Jointdnn: an efficient training and inference engine for intelligent mobile cloud computing services, *IEEE Trans. Mob. Comput.* (2019).
- [10] R. Hadidi, J. Cao, M.S. Ryoo, H. Kim, Towards collaborative inferencing of deep neural networks on internet of things devices, *IEEE Internet Things J.* (2020).
- [11] C. Hu, W. Bao, D. Wang, F. Liu, Dynamic adaptive dnn surgery for inference acceleration on the edge, in: *IEEE INFOCOM 2019-IEEE Conference on Computer Communications*, IEEE, 2019, pp. 1423–1431.
- [12] S. Yao, J. Li, D. Liu, T. Wang, S. Liu, H. Shao, T. Abdelzaher, Deep compressive offloading: speeding up neural network inference by trading edge computation for network latency, in: *Proceedings of the 18th Conference on Embedded Networked Sensor Systems*, 2020, pp. 476–488.
- [13] S. Laskaridis, S.I. Venieris, M. Almeida, I. Leontiadis, N.D. Lane, Spinn: synergistic progressive inference of neural networks over device and cloud, in: *Proceedings of the 26th Annual International Conference on Mobile Computing and Networking*, 2020, pp. 1–15.
- [14] J. Mao, Z. Yang, W. Wen, C. Wu, L. Song, K.W. Nixon, X. Chen, H. Li, Y. Chen, Mednn: A distributed mobile system with enhanced partition and deployment for large-scale dnn, in: *2017 IEEE/ACM International Conference on Computer-Aided Design, ICCAD, IEEE*, 2017, pp. 751–756.
- [15] Z. Zhao, K.M. Barijough, A. Gerstlauer, Deepthings: Distributed adaptive deep learning inference on resource-constrained iot edge clusters, *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* 37 (11) (2018) 2348–2359.
- [16] S. Teerapittayanon, B. McDanel, H.-T. Kung, Distributed deep neural networks over the cloud, the edge and end devices, in: *2017 IEEE 37th International Conference on Distributed Computing Systems, ICDCS, IEEE*, 2017, pp. 328–339.
- [17] E. Li, L. Zeng, Z. Zhou, X. Chen, Edge ai: On-demand accelerating deep neural network inference via edge computing, *IEEE Trans. Wireless Commun.* 19 (1) (2019) 447–457.
- [18] NVIDIA, Nvidia triton inference server, <https://developer.nvidia.com/nvidia-triton-inference-server>.
- [19] A. Dhakal, S.G. Kulkarni, K. Ramakrishnan, Gslice: controlled spatial sharing of gpus for a scalable inference platform, in: *Proceedings of the 11th ACM Symposium on Cloud Computing*, 2020, pp. 492–506.
- [20] D. Crankshaw, X. Wang, G. Zhou, M.J. Franklin, J.E. Gonzalez, I. Stoica, Clipper: A low-latency online prediction serving system, in: *14th {USENIX} Symposium on Networked Systems Design and Implementation ({NSDI} 17)*, 2017, pp. 613–627.
- [21] H. Shen, L. Chen, Y. Jin, L. Zhao, B. Kong, M. Philipose, A. Krishnamurthy, R. Sundaram, Nexus: A gpu cluster engine for accelerating dnn-based video analysis, in: *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, 2019, pp. 322–337.
- [22] A. Gujjarati, R. Karimi, S. Alzayat, W. Hao, A. Kaufmann, Y. Vigfusson, J. Mace, Serving {DNNs} like clockwork: Performance predictability from the bottom up, in: *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 2020, pp. 443–462.
- [23] ESnet, Lawrence Berkeley National Laboratory, iperf - the ultimate speed test tool for tcp, udp and sctp, <https://iperf.fr/>.

- [24] A. Krizhevsky, I. Sutskever, G.E. Hinton, Imagenet classification with deep convolutional neural networks, *Adv. Neural Inf. Process. Syst.* 25 (2012).
- [25] K. Simonyan, A. Zisserman, Very deep convolutional networks for large-scale image recognition, 2014, arXiv preprint [arXiv:1409.1556](https://arxiv.org/abs/1409.1556).
- [26] S. Zhang, Y. Li, X. Liu, S. Guo, W. Wang, J. Wang, B. Ding, D. Wu, Towards real-time cooperative deep inference over the cloud and edge end devices, *Proc. ACM Interactive Mobile Wearable Ubiquitous Technol.* 4 (2) (2020) 1–24.
- [27] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, Z. Wojna, Rethinking the inception architecture for computer vision, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 2818–2826.
- [28] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.
- [29] N. Dukkkipati, T. Refice, Y. Cheng, J. Chu, T. Herbert, A. Agarwal, A. Jain, N. Sutin, An argument for increasing tcp's initial congestion window, *SIGCOMM Comput. Commun. Rev.* 40 (3) (2010) 26–33, [http://dx.doi.org/10.1145/1823844.1823848](https://doi.org/10.1145/1823844.1823848).
- [30] N. Hu, P. Steenkiste, Evaluation and characterization of available bandwidth probing techniques, *IEEE J. Sel. Areas Commun.* 21 (6) (2003) 879–894, [http://dx.doi.org/10.1109/JSAC.2003.814505](https://doi.org/10.1109/JSAC.2003.814505).
- [31] NVIDIA, Multi-process service, https://docs.nvidia.com/deploy/pdf/CUDA_Multi_Process_Service_Overview.pdf.
- [32] A. Dhakal, S.G. Kulkarni, K.K. Ramakrishnan, Ecm1: Improving efficiency of machine learning in edge clouds, in: *2020 IEEE 9th International Conference on Cloud Networking (CloudNet)*, 2020, pp. 1–6, [http://dx.doi.org/10.1109/CloudNet51028.2020.9335804](https://doi.org/10.1109/CloudNet51028.2020.9335804).
- [33] F. Romero, Q. Li, N.J. Yadwadkar, C. Kozyrakis, INFaaS: Automated model-less inference serving, in: *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, USENIX Association, 2021, pp. 397–411.
- [34] U. Gupta, S. Hsia, V. Saraph, X. Wang, B. Reagen, G.-Y. Wei, H.-H.S. Lee, D. Brooks, C.-J. Wu, Deeprecsys: A system for optimizing end-to-end at-scale neural recommendation inference, in: *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture, ISCA, 2020*, pp. 982–995, [http://dx.doi.org/10.1109/ISCA45697.2020.00084](https://doi.org/10.1109/ISCA45697.2020.00084).
- [35] PyTorch, Torchserve, <https://pytorch.org/serve/>.
- [36] Tensorflow, Tensorflow serving, <https://www.tensorflow.org/tfx/guide/serving>.
- [37] C. Cox, D. Sun, E. Tarn, A. Singh, R. Kelkar, D. Goodwin, Serverless inferencing on kubernetes, 2020, CoRR abs/2007.07366. [arXiv:2007.07366](https://arxiv.org/abs/2007.07366).
- [38] M. Tan, Q.V. Le, Efficientnet: Rethinking model scaling for convolutional neural networks, 2019, arXiv preprint [arXiv:1905.11946](https://arxiv.org/abs/1905.11946).
- [39] PyTorch, Pytorch, <https://pytorch.org/>.
- [40] Intel, Intel xeon gold 5218 processor, <https://www.intel.com/content/www/us/en/products/sku/192444/intel-xeon-gold-5218-processor-22m-cache-2-30-ghz/specifications.html>.
- [41] NVIDIA, Nvidia triton perf analyzer, https://github.com/triton-inference-server/server/blob/master/docs/perf_analyzer.md.
- [42] NVIDIA, Jetson nano developer kit, <https://developer.nvidia.com/embedded/jetson-nano-developer-kit>.
- [43] Bert Hubert, Linux advanced routing & traffic control, <https://www.lartc.org/>.
- [44] R. Prasad, C. Dovrolis, M. Murray, K. Claffy, Bandwidth estimation: metrics, measurement techniques, and tools, *IEEE Network* 17 (6) (2003) 27–35, [http://dx.doi.org/10.1109/MNET.2003.1248658](https://doi.org/10.1109/MNET.2003.1248658).
- [45] E. Cuervo, A. Balasubramanian, D.-k. Cho, A. Wolman, S. Saroiu, R. Chandra, P. Bahl, Maui: Making smartphones last longer with code offload, in: *Proceedings of the 8th International Conference on Mobile Systems, Applications, and Services, MobiSys '10*, Association for Computing Machinery, New York, NY, USA, 2010, pp. 49–62, [http://dx.doi.org/10.1145/1814433.1814441](https://doi.org/10.1145/1814433.1814441).
- [46] G. Joshi, E. Soljanin, G. Wornell, Efficient redundancy techniques for latency reduction in cloud systems, *ACM Trans. Model. Perform. Eval. Comput. Syst.* 2 (2) (2017) [http://dx.doi.org/10.1145/3055281](https://doi.org/10.1145/3055281).
- [47] J. Huang, F. Qian, A. Gerber, Z.M. Mao, S. Sen, O. Spatscheck, A close examination of performance and power characteristics of 4 g lte networks, in: *Proceedings of the 10th International Conference on Mobile Systems, Applications, and Services*, 2012, pp. 225–238.